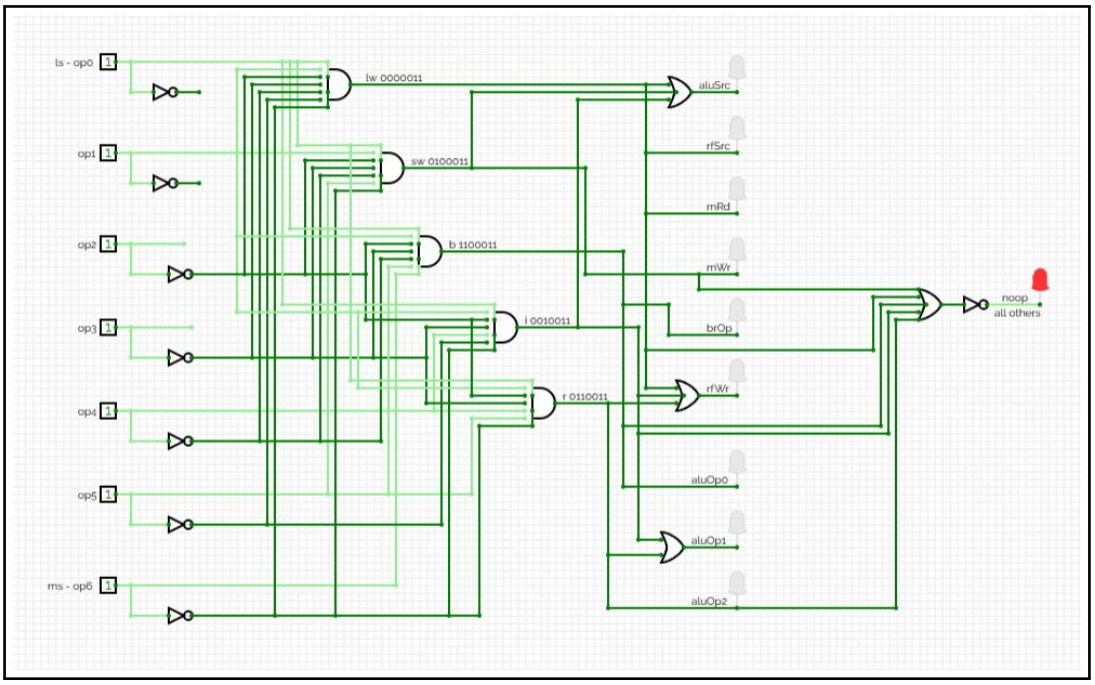
HY225 - Ασκήσεις 8 και 9 - csd5748

Άσκηση 8.

Άσκηση 8.5α:

Opcode	aluSrc	rfSrc	mRd	mWr	rfWr	brOp	aluOp
0000011	1	1	1	0	1	0	000
0100011	1	x	0	1	0	0	000
1100011	0	x	0	0	0	1	001
0010011	1	0	0	0	1	0	010
0110011	0	0	0	0	1	0	110
All others.	x	x	0	0	0	0	000

Πίνακας Αληθείας: Main Ctrl.

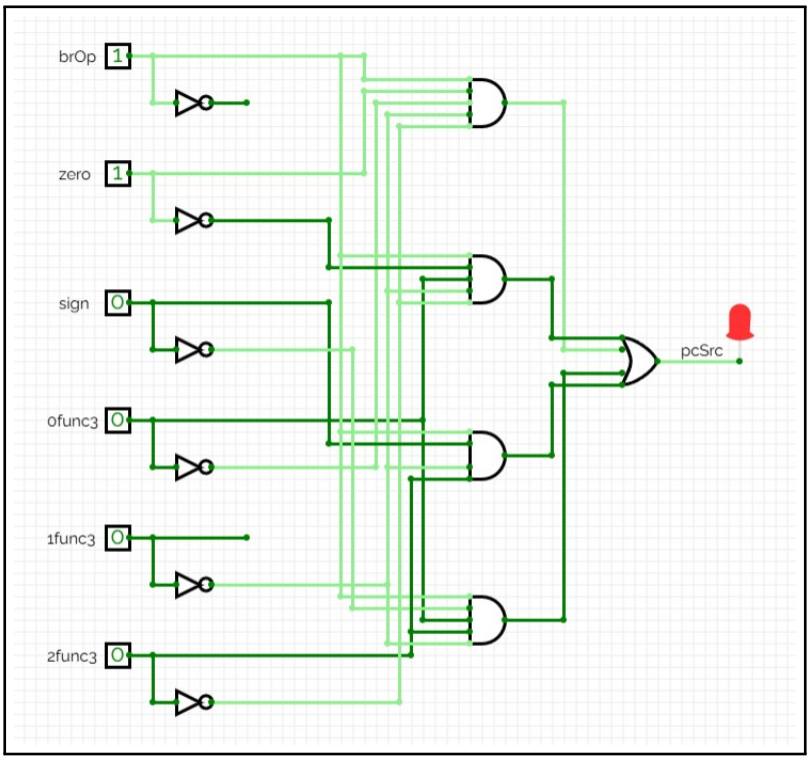


Κύκλωμα: Main Ctrl - σχεδιασμένο στο CircuitVerse.

Άσκηση 8.5β:

brOp	zero	sign	funct3	pcSrc
0	x	x	xxx	0
1	1	x	000	1
1	0	x	000	0
1	0	x	001	1
1	1	x	001	0
1	x	1	10x	1
1	x	0	10x	0
1	x	0	101	1
1	x	1	101	0

Πίνακας Αληθείας Διακλαδώσεων.

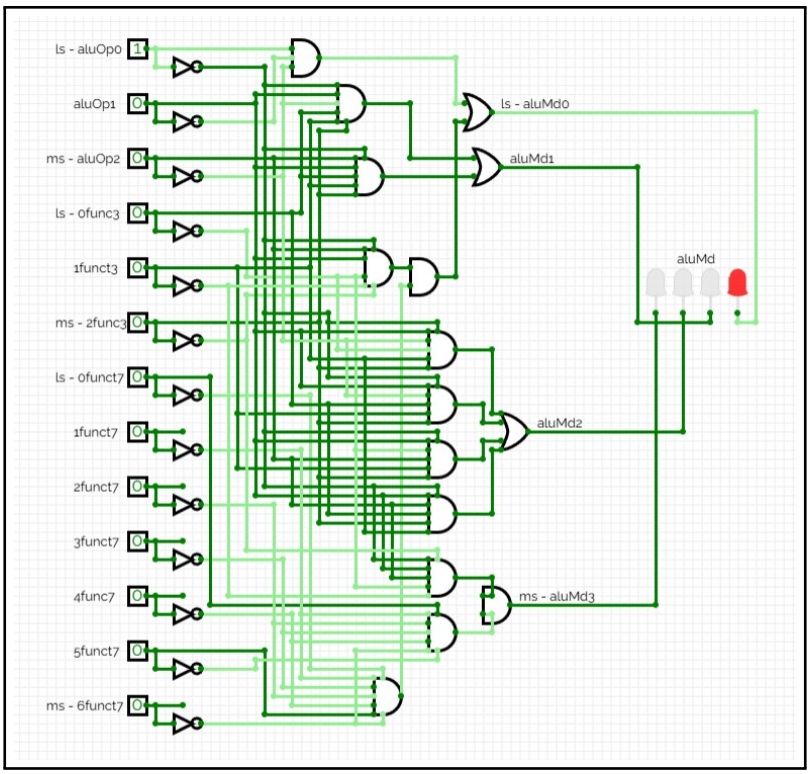


Κύκλωμα Σήματος Διακλαδώσεων - σχεδιασμένο στο CircuitVerse.

Άσκηση 8.5γ:

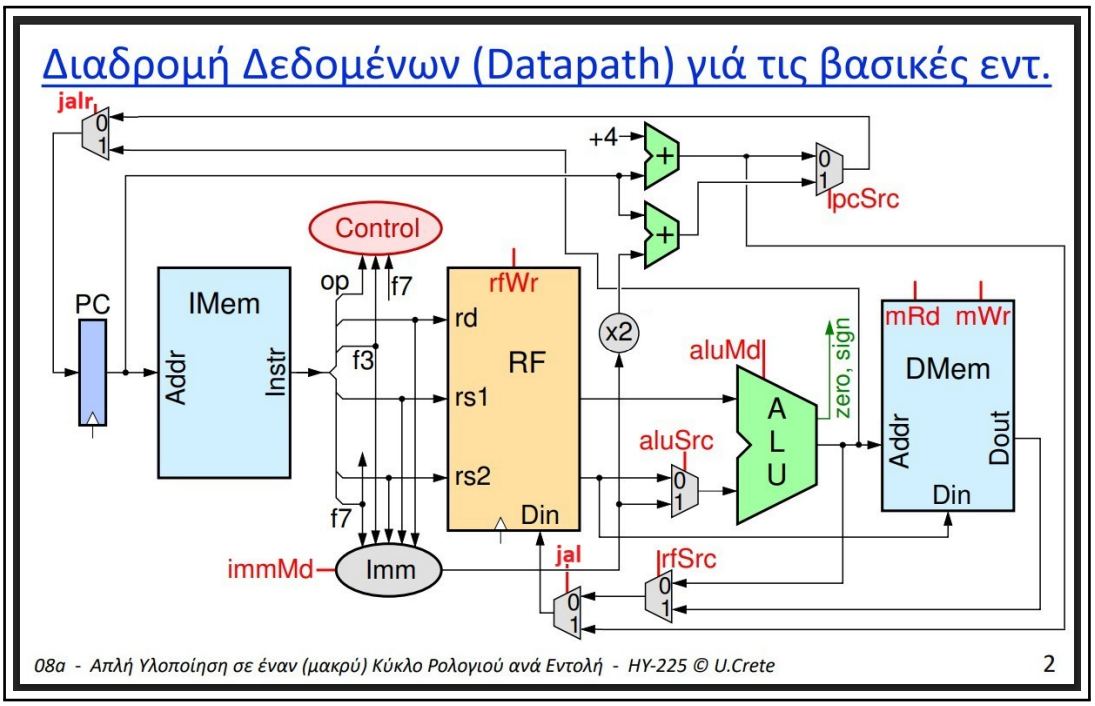
aluOp	funct3	funct7	aluMd
000	xxx	xxxxxxxx	00x0
001	xxx	xxxxxxxx	00x1
010	000	xxxxxxxx	00x0
010	110	xxxxxxxx	010x
010	111	xxxxxxxx	011x
010	other	xxxxxxxx	00x0
110	000	0000000	00x0
110	000	0100000	00x1
110	000	0000001	1xxx
110	000	other	00x0
110	110	xxxxxxxx	010x
110	111	xxxxxxxx	011x
110	other	xxxxxxxx	00x0

Πίναχας Αληθείας ALU Ctrl.



Κύκλωμα: ALU Ctrl - σχεδιασμένο στο CircuitVerse.

Άσκηση 8.6:



[Εικόνα από εδώ.]

Λοιπόν, τώρα που προσθέθηκαν δύο νέοι πολυπλέκτες στο κύκλωμα (πάνω-αριστερά και κάτω-κέντρο), θα πρέπει να προσθέσουμε και δύο νέα σήματα για τους ελέγχους των πολυπλεκτών (jal και jalr), καθώς και να τροποποιηθούν κάποια άλλα (rfWr και aluSrc), ώστε να λειτουργούν όλα όπως πρέπει. Έτσι, το νέο σήμα jal επιλέγει πια τιμη θα αποθηκευτεί στον καταχωρητή rd, η έξοδος της ALU ή η τιμή PC+4. Από την άλλη, το νέο σήμα ελέγχου jalr επιλέγει την επόμενη διεύθυνση PC, μεταξύ της εξόδου της ALU και του PC+4. Τέλος, το υπάρχον σήμα ελέγχου rfWr πρέπει να ενεργοποιείται και για τις δύο εντολές jal και jalr κατάλληλα.

Άσκηση 8.7:

Έχουμε ως Δεδομένα Καθυστερήσεων:
- Instruction Memory: 500 ps.
- Data Memory: 500 ps.
- Register File (read): 300 ps.
- Register File (write): 200 ps.
- ALU: 400 ps.
- Adder (PC): 400 ps.
- Control Logic: 200 ps.
- Multiplexers - PC: 0 ps **(αγνοούνται).**

a) Η εντολή: 500 ps.
b) Τα σήματα ελέγχου: 500 + 200 = 700 ps.
c) Οι τιμές των καταχωρητών πηγής: 500 + 300 = 800 ps.
d) Η έξοδος της ALU: 500 + 300 + 400 = 1200 ps.
e) Οι τιμές PC+4 και PC+2×Imm12: 400 ps και 500 + 400 = 900 ps αντίστοιχα.
f) Η τιμή γιά τον επόμενο PC: 500 + 300 + 400 = 1200 ps.
g) Η έξοδος της Data Memory: 500 + 300 + 400 + 500 = 1700 ps.
h) Η είσοδος Write Data του Αρχείου Καταχωρητών: 500 + 300 + 400 + 500 = 1700 ps.
i) Ολοκλήρωση τυχόν εγγραφής στο Αρχείο Κατ.: 500 + 300 + 400 + 500 + 200 = 1900 ps.

- Χειρότερη Περίπτωση: 1900 ps.
- Ελάχιστη περίοδος ρολογιού: 1.9 ns.
- Μέγιστη Συχνότητα Ρολογιού: f = 1 / 1.9 = 0.526 = 526 MHz
- Με Εντολή Διαίρεσης: 500 + 300 + 4400 + 500 + 200 = 5900 ps = 5.9 ns.
- Νέα συχνότητα: f = 1 / 5.9 = 0.169 = 169 MHz
- Τελικά, πιο αργό κατά: 526 / 169 = 3.112 φορές.

Άσκηση 9.

Άσκηση 9.9:

(1) Οι βαθμίδες (στάδια) της pipeline του επεξεργαστή (από τον Ripes) είναι:

α. Instruction Fetch (IF). Λειτουργία: Ανακτά την εντολή από την Instruction Memory (IM) χρησιμοποιώντας τη διεύθυνση του Program Counter (PC) και ενημερώνει τον PC κατά (PC+4) για την επόμενη εντολή και, τέλος, περνά την εντολή στο επόμενο στάδιο.

β. Instruction Decode (ID). Λειτουργία: Αποκωδικοποιεί την εντολή (opcode, registers, immediates) μέσω του Decoder καθώς και διαβάζει τους καταχωρητές πηγής (reg1, reg2) από το Register File (RF) και περνά τα δεδομένα στο επόμενο στάδιο.

γ. Execute (EX). Λειτουργία: Εκτελεί αριθμητικές ή λογικές πράξεις στην ALU με βάση τους (Op1, Op2) αλλά και υπολογίζει διευθύνσεις για load/store ή ελέγχει διακλαδώσεις (Branch), περνώντας το αποτέλεσμα στο επόμενο στάδιο.

δ. Memory Access (MEM). Λειτουργία: Πραγματοποιεί πρόσβαση στη Data Memory (DM) για εντολές load (ανάγνωση) ή store (εγγραφή), εαν χρειάζεται, και περνά τα δεδομένα ή τα αποτελέσματα στο επόμενο στάδιο.

ε. Write Back (WB). Λειτουργία: Εγγράφει το αποτέλεσμα από την ALU ή την DM πίσω στο Register File. Αυτό είναι και το τελευταίο στάδιο (βαθμίδα) της pipeline του συγκεκριμένου επεξεργαστή.

(2) Οι βαθμίδες (στάδια) της pipeline που είναι απαραίτητες για ολες τις εντολές είναι:

α. Instruction Fetch (IF), αφού ολες οι εντολές πρέπει να ανακτηθούν από τη μνήμη (Instruction Memory) για να ξεκινήσει η εκτέλεσή τους.

β. Instruction Decode (ID), αφού η αποκωδικοποίηση είναι απαραίτητη για να κατανοηθεί τι κάνει η εντολή και να διαβαστούν οι καταχωρητές, για να προετοιμαστούν για εκτέλεση.

γ. Execute (EX), αφού η ALU χρησιμοποιείται για όλες τις εντολές και είναι έτσι πάντα ενεργή.

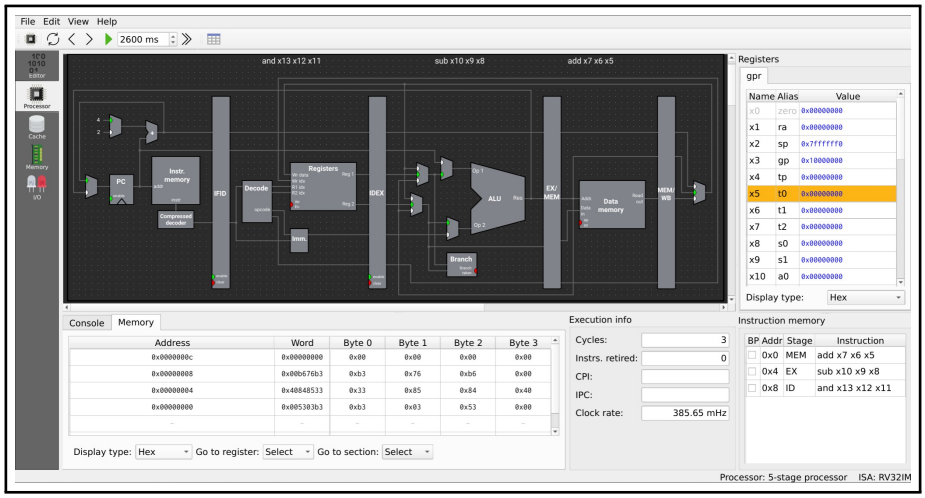
(3i) Οι βαθμίδες (στάδια) της pipeline που δεν χρειάζονται για εντολές αριθμητικών πράξεων:

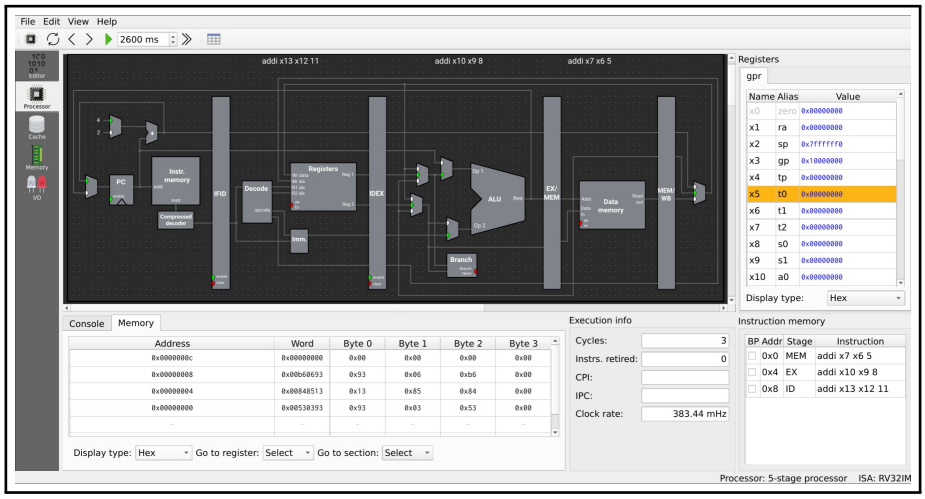
α. Memory Access (MEM), αφού οι αριθμητικές πράξεις δεν απαιτούν πρόσβαση στη μνήμη (Data Memory). Το αποτέλεσμα της ALU απλώς περνά στο επόμενο στάδιο.

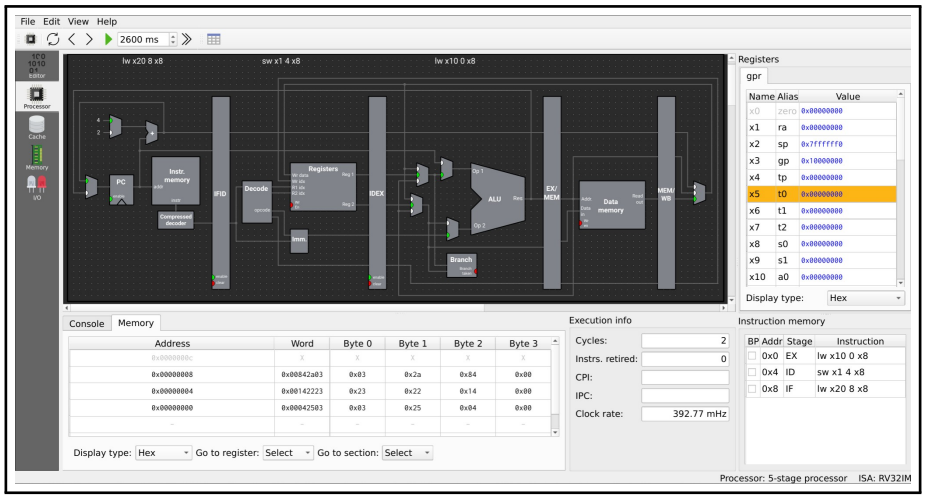
(3ii) Οι βαθμίδες (στάδια) της pipeline που δεν χρειάζονται για εντολές store είναι:

α. Write Back (WB), αφού οι εντολές αυτές δεν δημιουργούν αποτέλεσμα που να πρέπει να γραφτεί σε κάποιο καταχωρητή.

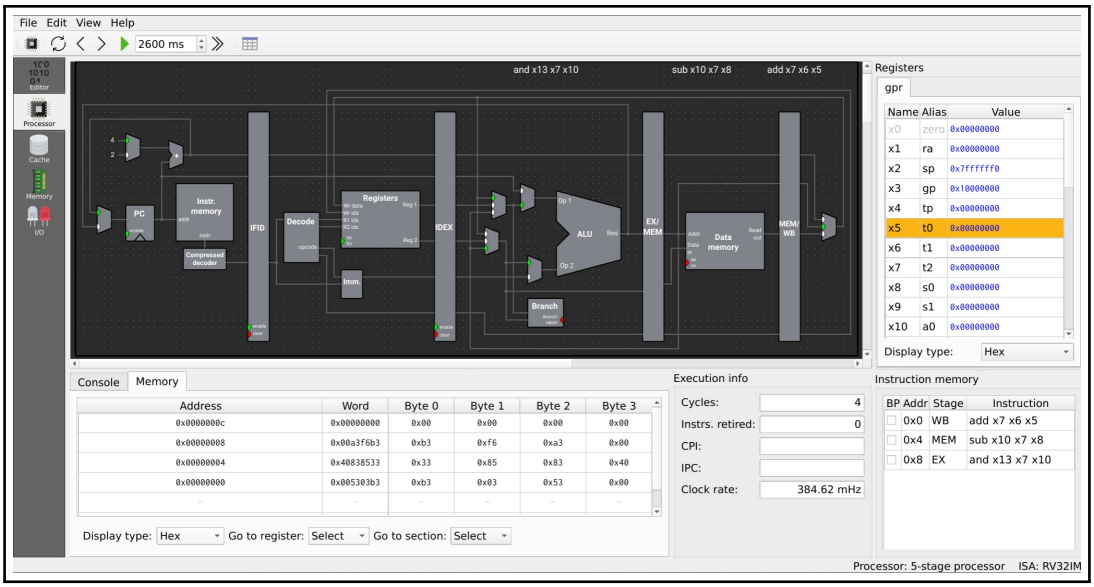
(4) **Οι αλληλουχίες εντολών χωρίς αλληλεξαρτήσεις και οι εκτελέσεις τους στον Ripes:**



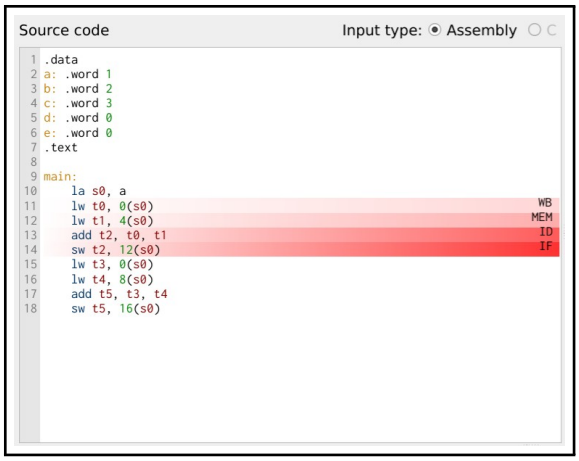


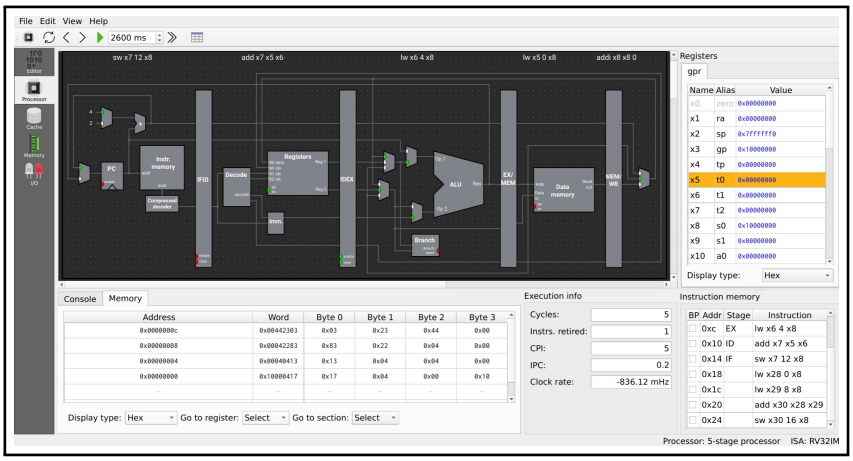


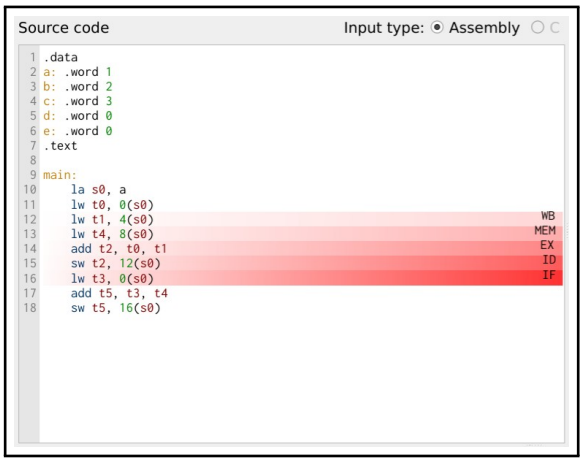
(5) Οι νέες εξαρτημένες εντολές του ερ. (4α) και οι εκτελέσεις τους στον Ripes (μαζί):

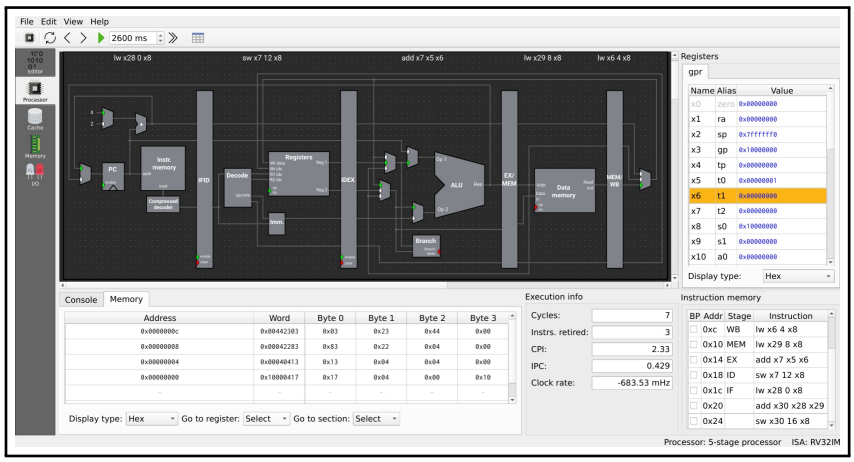


(6) Οι εντολές πριν και μετά την αναδιάταξη μαζί με τις εκτελέσεις τους στον Ripes είναι:









(7) Οι περισσότεροι από τους 189 - [5 + (125 - 1)] - 6 = 54 χαμένους κύκλους οφείλονται σε προβλήματα των 16 διακλαδώσεων στο βρόχο αλλά και στις κλήσεις συναρτήσεων που χρησιμοποιούνται στο factorial.s. Μία, λοιπόν, τεχνική για να βελτιωθεί αυτό είναι η επόμενη εντολή μετά τη διακλάδωση να εκτελείται πάντα. Με αυτόν τον τρόπο, θα γλιτώναμε περίπου 16 κύκλους.

ΤΕΛΟΣ